

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

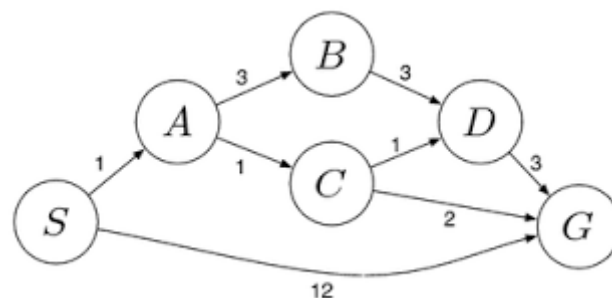
Signature of the student:

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Annexure A - Answers **Analytical Problem Solving**

1. Water Jug Problem (4-gallon and 3-gallon jugs, target: 2 gallons in the 4-gallon jug)

Problem understanding. State = (x, y), where x = water in the 4-gallon jug (0-4) and y = water in the 3-gallon jug (0-3). Initial state (0, 0). Goal state (2, n) for any n.

Production rules (operators):

No.	Rule	Condition
1	(x, y) → (4, y) Fill the 4-gal jug from the pump	$x < 4$
2	(x, y) → (x, 3) Fill the 3-gal jug from the pump	$y < 3$
3	(x, y) → (0, y) Empty the 4-gal jug on the ground	$x > 0$
4	(x, y) → (x, 0) Empty the 3-gal jug on the ground	$y > 0$
5	(x, y) → (4, y - (4 - x)) Pour from 3-gal into 4-gal until the 4-gal is full	$x + y \geq 4, y > 0$
6	(x, y) → (x - (3 - y), 3) Pour from 4-gal into 3-gal until the 3-gal is full	$x + y \geq 3, x > 0$
7	(x, y) → (x + y, 0) Pour all of the 3-gal into the 4-gal	$x + y \leq 4, y > 0$
8	(x, y) → (0, x + y) Pour all of the 4-gal into the 3-gal	$x + y \leq 3, x > 0$

Solution (shortest path found by BFS - 6 steps):

Step	State (4-gal, 3-gal)	Action	Rule
0	(0, 0)	Start	-
1	(0, 3)	Fill the 3-gallon jug from the pump	2
2	(3, 0)	Pour all of the 3-gal jug into the 4-gal jug	7
3	(3, 3)	Fill the 3-gallon jug again	2
4	(4, 2)	Pour from 3-gal into 4-gal until the 4-gal is full (1 gal moves, 2 gal remain)	5
5	(0, 2)	Empty the 4-gallon jug on the ground	3
6	(2, 0)	Pour the 2 gallons from the 3-gal jug into the 4-gal jug	7

Result: The 4-gallon jug contains exactly **2 gallons** after 6 moves. (An equally short alternative: (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3), which also leaves 2 gallons in the 4-gallon jug.)

2. Mars Rover - PEAS Description

i. Percepts of the agent

- Camera images (panoramic / stereo / close-up) of terrain, rocks and soil
- Spectrometer, X-ray and chemical-analysis readings of rock and soil samples
- Temperature, pressure, wind and radiation (weather) sensor readings
- Position and motion data: wheel encoders, IMU (tilt/orientation), odometry, sun sensor
- Terrain data: slope, wheel slip, obstacle proximity (hazard cameras / LIDAR)
- Internal status: battery level, solar-panel power, component temperature, fault flags
- Commands and updates received from Earth mission control (with a communication delay)

ii. Operating environment (characterization)

Property	Value	Reason
Observable	Partially observable	Sensors see only the nearby terrain; the hidden composition of rocks and the whole map are unknown
Agents	Single agent (mostly)	The rover works alone; Earth control only issues high-level commands
Deterministic?	Stochastic	Wheel slip, dust, sand and weather make action outcomes uncertain
Episodic?	Sequential	Current movement and sampling decisions affect energy, position and future options

Property	Value	Reason
Static?	Dynamic	Dust storms, temperature and light change while the rover is deliberating
Discrete?	Continuous	Position, time, temperature and sensor values are continuous
Known?	Largely unknown	Terrain and sample properties are not known in advance

iii. Actions the agent can take

- Drive: move forward / backward, turn left / right, stop, climb or avoid obstacles (steer wheels)
- Position and extend the robotic arm; use the drill or scoop to collect rock / soil samples
- Operate analysis instruments (spectrometer, microscope, camera, chemical lab) on a sample
- Take photographs and record measurements
- Store, compress and transmit data to Earth (via orbiter or antenna)
- Manage energy: recharge using solar panels, enter sleep / heating mode, shut down non-essential systems
- Deploy or retract instruments; clean dust off panels / lenses

iv. Performance measure

- Number, variety and scientific value of samples collected and analysed correctly
- Area of Mars explored and quality / quantity of data successfully transmitted to Earth
- Safety: no collisions, falls or stuck wheels; rover survives for the full mission duration (longevity)
- Efficiency: energy and time used per sample / per kilometre travelled

v. Most suitable agent architecture

A **utility-based, goal-based agent with an internal model (model-based) and learning component** - i.e., a hybrid / layered architecture - is most suitable. Reasons:

- **Partial observability** requires an internal state / world model (map, battery, position) - a simple reflex agent is not enough.
- **Communication delay** with Earth (several to 20+ minutes) means the rover must act autonomously towards goals (reach a rock, collect a sample, analyse it).
- **Conflicting objectives** (science value vs. safety vs. energy vs. time) must be traded off, which needs a utility function to choose the best action.
- **Unknown, changing terrain** benefits from a learning element that improves navigation and sample selection over time.
- A low-level **reflex layer** is still kept for emergencies (e.g., stop immediately at a cliff edge or on a steep slope).

3. 8-Queens Problem - Problem Formulation and Search

Problem understanding. Place 8 queens on an 8 x 8 board so that no two queens share a row, column or diagonal. The configuration in the figure is **not a solution** because the queen in the first column and the queen in the last column lie on the same diagonal, so they attack each other. Only the final arrangement matters, not the path - so this is a *constraint-satisfaction style* problem suited to uninformed search with backtracking.

Incremental formulation (problem components):

Component	Definition
States	Any arrangement of 0 to 8 queens on the board. (Improved version: n queens placed, one per column in the leftmost n columns, with no queen attacked.)
Initial state	Empty board - no queens placed
Actions	Add a queen to an empty square. (Improved: add a queen to the leftmost empty column in a square that is not attacked by any existing queen.)
Transition model	Returns the board with a queen added to the specified square
Goal test	8 queens are on the board and none is attacked by another
Path cost	Not relevant (every step costs 1; only the final configuration matters)

Size of the search space:

Formulation	Number of states
Naive: place any queen on any empty square	$64 \times 63 \times \dots \times 57 = \text{about } 1.8 \times 10^{14}$
Complete-state: 8 queens, one per column, any row	$8^8 = 16,777,216$
Improved incremental: one queen per column, non-attacking	only 2,057 states

Search strategy. All solutions lie at depth 8 (one queen per column), so **Depth-First Search with backtracking** is the best uninformed choice: it needs only $O(bm)$ memory, and it backtracks as soon as a column has no safe row. BFS also finds a solution but stores a huge frontier. Checking attacks: two queens $(r1, c1)$ and $(r2, c2)$ attack if $r1 = r2$ or $|r1 - r2| = |c1 - c2|$.

Example valid solution (row number of the queen in columns 1-8): **1, 5, 8, 6, 3, 7, 2, 4** - no two share a row, and all diagonals (row-col and row+col) are distinct. The problem has 92 solutions in total (12 distinct up to rotation and reflection). The given configuration is rejected by the goal test, and the search would backtrack from it.

4. OLA Cab Booking - Problem-Solving Agent and Pseudocode

Type of agent. A **goal-based problem-solving agent** (with a utility-based selection of the cab type). The agent has a clear goal - reach the destination - and the environment is observable, discrete and deterministic enough to formulate the problem, search for a plan (route and cab type) and then execute it. The choice among mini / micro / sedan / shared / prime is made by the utility (fare, ETA, comfort) function.

Problem formulation:

Component	Definition
Initial state	Customer at pickup location, no cab booked
Goal state	Customer dropped at the destination
Actions	Enter pickup/drop, view cab types, select cab type, confirm booking, track cab, board, ride, pay
Transition model	Each action moves the booking from one status to the next (requested -> assigned -> arrived -> on trip -> completed)
Path cost / utility	Weighted combination of fare, waiting time, travel time and comfort preference

Pseudocode (simple problem-solving agent):

```

function OLA-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty), state, goal, problem

  state <- UPDATE-STATE(state, percept)      // pickup, drop, time, traffic
  if seq is empty then
    goal   <- FORMULATE-GOAL(state)          // reach destination D from P
    problem <- FORMULATE-PROBLEM(state, goal)
    seq    <- SEARCH(problem)                // route + cab selection
    if seq = failure then return NO-OP        // no cab available
  action <- FIRST(seq)
  seq    <- REST(seq)
  return action

function SEARCH(problem) returns a booking plan
  options <- GET-AVAILABLE-CABS(pickup, drop) // mini, micro, sedan,
                                              // shared, prime ...

  if options is empty then return failure
  for each cab in options do
    cab.fare   <- ESTIMATE-FARE(cab, route)
    cab.eta    <- ESTIMATE-ARRIVAL(cab)
    cab.utility <- w1*comfort(cab) - w2*cab.fare - w3*cab.eta
  best <- cab in options with maximum utility (or the one chosen by user)
  return [CONFIRM-BOOKING(best), TRACK(best), BOARD, RIDE, PAY]

main:
  state <- INITIAL-STATE(pickup, drop)
  while not GOAL-TEST(state) do           // goal: customer reached destination
    percept <- GET-PERCEPT()
    action  <- OLA-AGENT(percept)
    EXECUTE(action)
    state   <- RESULT(state, action)
  END

```

5. Uniform Cost Search (UCS) - Least-cost path from S to G

Problem understanding. Directed weighted graph with edges: S-A = 1, S-G = 12, A-B = 3, A-C = 1, B-D = 3, C-D = 1, C-G = 2, D-G = 3. UCS expands the frontier node with the lowest path cost $g(n)$ using a priority queue; the goal test is applied when a node is **selected for expansion**, which guarantees an optimal path.

Step-by-step execution (frontier shown as node:cost, sorted by cost):

Step	Node expanded	Frontier after expansion	Explored set
1	S (0)	A:1, G:12	S
2	A (1)	C:2, B:4, G:12	S, A
3	C (2)	D:3, B:4, G:4 (via C, replaces G:12)	S, A, C
4	D (3)	B:4, G:4 (G via D costs $6 > 4$, discarded)	S, A, C, D
5	B (4)	G:4 (D via B costs $7 > 3$, discarded)	S, A, C, D, B
6	G (4) - goal selected	Stop	S, A, C, D, B, G

Candidate paths to G:

Path	Total cost
S -> G	12
S -> A -> B -> D -> G	$1 + 3 + 3 + 3 = 10$
S -> A -> C -> D -> G	$1 + 1 + 1 + 3 = 6$
S -> A -> C -> G	$1 + 1 + 2 = 4$ (minimum)

Final answer: The least-cost route is **S -> A -> C -> G with total cost 4** (cost units = fuel + time). Order of node expansion: S, A, C, D, B, G (B and G tie at cost 4; either order gives the same optimal answer).

Interpretation. The direct route S -> G looks short in hops but costs 12, three times more than the optimal route. UCS is **complete** (all step costs are positive) and **optimal** (it always expands the cheapest node first). The company should therefore route packages through warehouses A and C.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem